Assignment 3:
Block Matching and Dynamic Programming

Ezz Eldeen 9114,
Nour Essam Zeidan 8841,
Farah Bassiouny 8610

Abstract

This report details the implementation and comparison of two stereo vision algorithms for calculating depth via disparity maps: Block Matching (a greedy sliding window approach) and Dynamic Programming (a global scanline optimization approach). While Block Matching provides a baseline understanding of pixel matching, this report emphasizes the Dynamic Programming approach, which successfully models physical occlusions and significantly improves depth map accuracy at object boundaries.

1 Introduction

In stereo vision, 3D depth can be estimated from two 2D images (Left and Right) taken from slightly different horizontal positions. Because the cameras are offset, objects closer to the lens will shift horizontally more than objects further away. This horizontal shift is called **disparity**. By finding exactly which pixel in the Left image corresponds to which pixel in the Right image, we can calculate the disparity and construct a dense 3D depth map of the scene.

2 Part 1: The Block Matching Approach

The Block Matching algorithm is a local search method. For a given target pixel in the Left image, we define a search window (e.g., 1×1, 5×5, or 9×9) around it. We then slide an identically sized window across the corresponding scanline in the Right image.

2.1 Cost Calculation

At each step of the slide, we calculate the difference between the two windows using either the Sum of Absolute Differences (SAD) or the Sum of Squared Differences (SSD). The window position that yields the absolute lowest error score is declared the match. The distance the window had to shift to find this match is recorded as the disparity.

2.2 Visualizations: Window Sizes

Figure 1: Block Matching Disparity Map (Window Size 1×1). Notice the high level of noise due to a lack of spatial context.

Figure 2: Block Matching Disparity Map (Larger Window). The noise is reduced, but object boundaries become blurred (the "foreground fattening" effect).

2.3 Limitations of Block Matching

While larger windows smooth out noise, Block Matching possesses a fatal flaw: it is completely blind to **occlusions**. When a foreground object eclipses the background, certain pixels physically cannot be seen by both cameras. A sliding window algorithm will stubbornly force a match anyway, resulting in highly inaccurate depth values around the edges of objects.

3 Part 2: The Dynamic Programming Approach

To solve the occlusion problem, we pivot from a local greedy search to a global optimization strategy using Dynamic Programming (DP). Instead of guessing the best match window by window, DP evaluates every single possible timeline of how an entire row of pixels could align, strictly penalizing bad matches while allowing the algorithm to mathematically "skip" occluded pixels.

3.1 The State Space and Penalty System

We construct an $N \times N$ matrix where the vertical axis represents the Left scanline and the horizontal axis represents the Right scanline. A path through this matrix dictates the physical reality of the scene:

- **Diagonal Step:** Both cameras see the object. This is a *Match*.
- **Vertical/Horizontal Step:** One camera's view is blocked by a foreground object. This is an *Occlusion*.

To find the optimal path, we utilize a penalty system balancing the Match Cost and the Skip Penalty:

$$D(i,j) = \min \begin{cases} D(i-1,j-1) + \frac{(L_i - R_j)^2}{\sigma^2} & \text{(Match)} \\ D(i-1,j) + c_0 & \text{(Skip Left)} \\ D(i,j-1) + c_0 & \text{(Skip Right)} \end{cases} \tag{1}$$

3.2 Tuning the Algorithm: σ and c_0

The parameters c_0 (set to 1) and σ (set to 2) act as the deciding factors between matching and occluding.

- **The Skip Penalty (c_0):** Represents the flat cost of admitting a pixel is hidden.
- **The Forgiveness Dial (σ):** Accounts for natural camera noise. Because $\sigma^2 = 4$, a color difference between the Left and Right pixel of up to 2 shades results in a match penalty of ≤ 1. If the color difference exceeds 2 shades, the penalty surpasses c_0, and the algorithm mathematically prefers to register an occlusion.

3.3 Backtracking and Disparity Extraction

Once the matrix is populated with the accumulated minimum costs, we trace the optimal path backward from (N, N) to $(0, 0)$. Every diagonal step on this winning path confirms a physical match, allowing us to calculate the depth: Disparity $= |i - j|$. Every vertical step confirms a blind spot for the Right camera, prompting us to log a disparity of 0 for that pixel.

3.4 Visualizations: DP Results

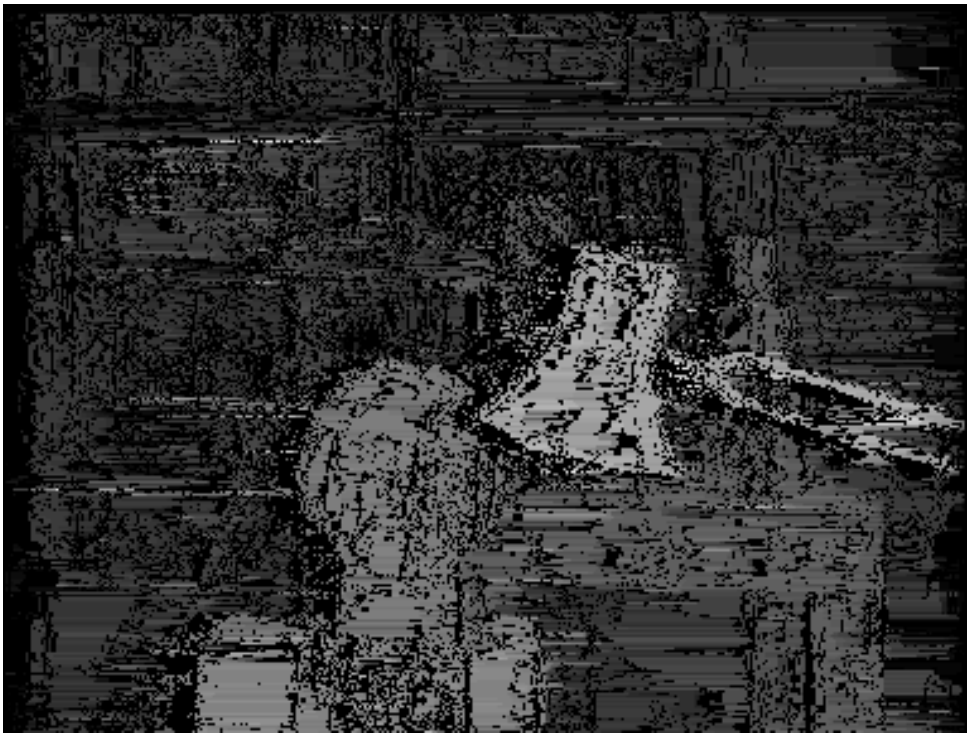

Figure 3: Dynamic Programming Disparity Map. Notice the sharp, accurate object boundaries and the clean handling of occluded regions (marked as 0 disparity).

3.5 Bonus: Parameter Analysis Graphs

As part of the extended analysis, the following graphs demonstrate the relationship between the algorithm parameters and the resulting error rates.

L \ R	4	2	8	4	0
4	0	1	2	3	4
6	1	2	3	4	5
2	2	1	2	3	4
8	3	2	1	2	3
4	4	3	2	1	2

Figure 4: Most Optimal Path in example row

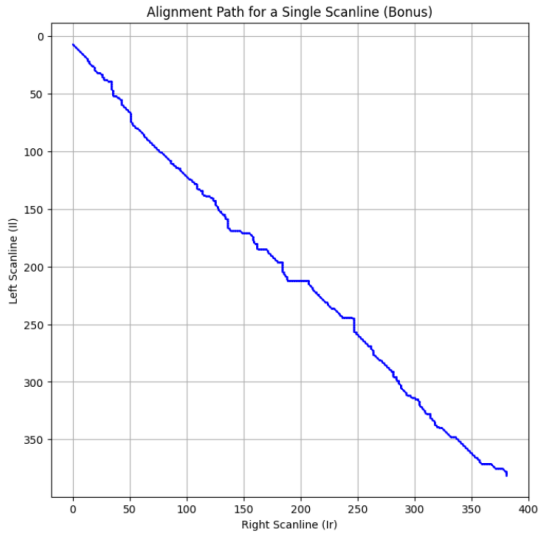

Figure 5: Analysis of actual scanline.

4 Conclusion

The transition from Block Matching to Dynamic Programming illustrates the importance of modeling physical constraints in computer vision. By translating the real-world geometry of occlusions into a mathematical penalty system, Dynamic Programming effectively filters out impossible matches. This results in a vastly superior disparity map, characterized by crisp object boundaries and an accurate representation of 3D depth.